Prompt 4.md

KKC TEXT ADVENTURE — PROMPT 4 OF 180
==========
Kilvin, Ambrose, and Social Consequence
==========

This prompt builds directly on Prompts 1, 2, and 3.
Do not restructure, rename, or refactor anything from those prompts.
Only add to what exists.

==========
HARD BANS — UNCHANGED
==========

No external APIs. No HTTP clients. No LLM. No Anthropic, OpenAI, or any remote service. No web server. No frontend. Fully offline only.

No new abstractions unless explicitly required here.
No new files unless listed in this prompt.
Do not refactor working code from earlier prompts.

==========
CANON CHARACTER RULES — READ FIRST
==========

Kilvin is not a kindly generic mentor. He is practical, grave, exacting, safety-minded, and morally serious about the reputation of arcanists. He can praise skill, but he does not excuse poor judgment. He values care, discipline, and consequences over cleverness alone.

Ambrose is not a cartoon villain. He is petty, arrogant, entitled, socially dangerous, and publicly cutting. His power comes less from violence than from status, humiliation, pressure, and his willingness to make life unpleasant for people beneath him.

Do NOT:
- make Kilvin soft, chatty, or overly available
- make Ambrose witty in a modern way
- turn either NPC into a quest dispenser
- invent large new canon events
- script sabotage, malfeasance, or high-drama confrontations yet
- resolve any long-form rivalry arc in this prompt

Keep this prompt grounded in small-scale University consequences: approval, rebuke, work, public tension, and reputation.

==========

GOAL OF THIS PROMPT

═══

By the end of Prompt 4, the player should be able to:

- Encounter Kilvin at the Fishery and speak to him in a way that feels canon-true
- Ask Kilvin for work and either be approved or refused based on condition, judgment, and context
- Work a Fishery shift for modest pay if approved
- Encounter Ambrose at the Archives exterior and feel social pressure
- Choose how to respond to Ambrose: ignore him, answer carefully, or answer sharply
- See those choices affect social standing and NPC trust
- Understand that University life is shaped not only by money and sympathy, but also by authority, status, and reputation

═══

SCOPE OF THIS PROMPT

═══

This prompt covers:

1. Add Kilvin and Ambrose as seeded NPCs
2. Extend npcProfiles.ts with Kilvin and Ambrose
3. Add FisheryState to PlayerState
4. Add socialEngine.ts for Fishery work approval and Ambrose response
5. Add authored dialogue pools for Kilvin and Ambrose
6. Add commands:
   ask kilvin for work
   work fishery
   ignore ambrose
   answer ambrose carefully
   answer ambrose sharply
7. Apply modest reputation/trust consequences
8. Reset Fishery approval daily
9. Add tests for Kilvin work approval and Ambrose responses

This prompt does NOT cover:

- Sygaldry (Prompt 7)
- Naming
- Ambrose sabotage, theft, candle plots, plum bob, or later major arcs
- Kilvin project review or artificing project submission
- Master-level examinations
- Eolian / Imre (Prompt 5)
- Academic rank advancement (Prompt 6)

═══

NEW FILES IN THIS PROMPT

═══

Add exactly these files:

```
src/
  engine/
    socialEngine.ts          ← Kilvin work approval + Ambrose response logic
  content/
    authorityDialogue.ts     ← authored response pools for Kilvin/Ambrose
tests/
  socialEngine.test.ts
```

Modify these existing files:
- src/types/index.ts
- src/engine/state.ts
- src/engine/actions.ts
- src/engine/time.ts
- src/engine/npcEngine.ts
- src/content/npcProfiles.ts
- src/db/seed.ts

Do not add any other files.

═══
TYPES — additions to src/types/index.ts
═══

Add these interfaces. Do not remove or change existing ones.

```
export interface FisheryState {
  approved_today: boolean;
  shifts_completed_today: number;
  last_approval_day: number | null;
}
```

Also add to PlayerState:
```
  fishery_state: FisheryState;
```

Update initDefaultPlayerState() in engine/state.ts to include:
```
  fishery_state: {
    approved_today: false,
    shifts_completed_today: 0,
    last_approval_day: null,
  }
```

Also update the default reputation.npc_trust object in initDefaultPlayerState() to include:
```
  simmon: 60
```

wilem: 55
anker: 50
kilvin: 45
ambrose: 20

These are baseline relationship values for MVP.
Do not expose the raw numbers directly in normal narration.

═══

SEED UPDATE — src/db/seed.ts

═══

Keep all existing locations and NPCs unchanged.
Add exactly these 2 NPCs using INSERT OR IGNORE:

kilvin
name: "Kilvin"
location_id: university_fishery_outer
era: university
temperament: "grave, methodical, practical, morally serious"
speech_style: "measured, formal, heavily accented Aturan,
sparse praise, direct disapproval when warranted"

ambrose
name: "Ambrose"
location_id: university_archives_exterior
era: university
temperament: "arrogant, cutting, entitled, socially dangerous"
speech_style: "coldly amused, status-conscious, dismissive,
sharp in public, cruel when crossed"

Do not add new locations in this prompt.

═══

NPC PROFILES — update src/content/npcProfiles.ts

═══

Extend NPC_PROFILES with: kilvin, ambrose

Rules for all text:
- Original prose only
- Do NOT reproduce Rothfuss lines
- Voice must match canon
- Keep each line concise and playable
- greeting_pool: exactly 3 entries
- exit_lines: exactly 2 entries

Kilvin profile:

id: 'kilvin'
name: 'Kilvin'
era: 'university'
home_location_id: 'university_fishery_outer'
temperament: 'grave, methodical, practical, morally serious'
speech_style: 'formal, precise, accented Aturan, judges skill and judgment together'

known_topics:
- artificing
- sympathy safety
- work
- materials
- discipline
- University reputation

taboo_topics:
- private life
- gossip
- naming
- noble politics

greeting_pool:
exactly 3 brief greetings that feel like Kilvin
He should sound occupied, perceptive, and not unkind.

exit_lines:
exactly 2 brief ways Kilvin ends a conversation.
They should feel practical rather than warm.

Ambrose profile:
id: 'ambrose'
name: 'Ambrose'
era: 'university'
home_location_id: 'university_archives_exterior'
temperament: 'arrogant, cutting, entitled, socially dangerous'
speech_style: 'smooth when it suits him, contemptuous when crossed, public-facing cruelty'

known_topics:
- rank
- admissions
- the Archives
- nobles
- student gossip
- Kvothe

taboo_topics:
- apology
- weakness

- consequences for his own behaviour

greeting_pool:
exactly 3 brief hostile or condescending greetings.
None should be cartoonish.

exit_lines:
exactly 2 ways Ambrose dismisses the conversation.

Do not flatten either character into a generic RPG NPC.

═══

AUTHORED DIALOGUE — src/content/authorityDialogue.ts

═══

Export these constants:

KILVIN_WORK_APPROVAL: string[]
KILVIN_WORK_DENIAL_UNFIT: string[]
KILVIN_WORK_DENIAL_NOT_HERE: string[]
KILVIN_WORK_ALREADY_APPROVED: string[]
KILVIN_WORK_SHIFT_COMPLETE: string[]

AMBROSE_IGNORE_LINES: string[]
AMBROSE_CAREFUL_RESPONSE_LINES: string[]
AMBROSE_SHARP_RESPONSE_LINES: string[]
AMBROSE_NOT_HERE_LINES: string[]

Rules:
- Original prose only
- Second person present tense where appropriate
- Maximum 2 sentences per entry
- Exactly 3 entries in each pool
- Deterministic selection only:
  index = state.day_number % 3
- Kilvin lines should be practical, restrained, and safety-minded
- Ambrose lines should sting socially, not become melodramatic

Specific tone guidance:

Kilvin approval:
He approves work because there is honest labour to be done, not because he is indulgent.

Kilvin denial:
He refuses if the player is too exhausted, too cold, visibly injured, or otherwise unsafe to work.

Ambrose responses:
  The response should reflect the player's chosen stance:
    ignore → controlled refusal to engage
    careful → measured, socially careful answer
    sharp   → cleverness with a cost

Do not let any line sound modern or jokey.

═══

ENGINE — src/engine/socialEngine.ts

═══

This file owns Fishery approval/work logic and Ambrose response logic.
It is engine-only. It does not call narrator methods.

Implement and export:

───

getNpcTrust(state: PlayerState, npc_id: string): number

───

Return state.reputation.npc_trust[npc_id] if present.
If missing, return 50.

───

setNpcTrust(
  state: PlayerState,
  npc_id: string,
  value: number
): PlayerState

───

Return a new PlayerState with the npc_trust value clamped 0–100.

───

adjustUniversitySocial(
  state: PlayerState,
  delta: number
): PlayerState

───

Return a new PlayerState with university_social adjusted and clamped 0–100.

───

askKilvinForWork(
  state: PlayerState,
  db
): { message: string; newState: PlayerState }

Rules:
  - Player must be at university_fishery_outer
  - Kilvin must be present there
  - If not at the Fishery or Kilvin absent:
      return denial line from KILVIN_WORK_DENIAL_NOT_HERE
      state unchanged
  - If state.fatigue > 80, or state.warmth < 25, or state.injuries.length > 0:
      return denial line from KILVIN_WORK_DENIAL_UNFIT
      reduce Kilvin trust by 1 only if the player is insisting while clearly unfit
      state otherwise unchanged
  - If state.fishery_state.approved_today is already true:
      return line from KILVIN_WORK_ALREADY_APPROVED
      state unchanged
  - Otherwise:
      set fishery_state.approved_today = true
      set fishery_state.last_approval_day = state.day_number
      increase Kilvin trust by 2
      return line from KILVIN_WORK_APPROVAL

Kilvin is approving a shift, not granting friendship.

```
workFisheryShift(
  state: PlayerState,
  db
): { message: string; newState: PlayerState }
```

Rules:
  - Player must be at university_fishery_outer
  - Kilvin does not need to be re-asked if approved_today is true
  - If approved_today is false:
      return "Kilvin has not set you to work."
      state unchanged
  - If shifts_completed_today >= 2:
      return line from KILVIN_WORK_SHIFT_COMPLETE
      state unchanged

On a successful Fishery shift:
  - deterministic pay:
      pay = 2 + ((state.day_number + state.fishery_state.shifts_completed_today) % 2)
      // results in 2 or 3 drabs
  - call advanceTime(state, 1)
  - then apply:
      money_drabs += pay
      fishery_state.shifts_completed_today += 1

fatigue += 12 (clamp max 100)
hunger += 8 (clamp max 100)
warmth += 5 (clamp max 100)   // the Fishery is hot work
academic_standing += 1 (clamp max 100)
Kilvin trust += 1 (clamp max 100)

Return a short in-world line about dull, careful work and the modest pay.
Do not romanticise the labour.

---

respondToAmbrose(
  mode: 'ignore' | 'careful' | 'sharp',
  state: PlayerState,
  db
): { message: string; newState: PlayerState }

Rules:
- Player must be at the same location as Ambrose
- For MVP, Ambrose is only present if his npc row location_id matches state.location_id
- If Ambrose is not present:
  return a line from AMBROSE_NOT_HERE_LINES
  state unchanged

Mode effects:

ignore:
- return line from AMBROSE_IGNORE_LINES
- Ambrose trust -= 2
- no change to university_social

careful:
- return line from AMBROSE_CAREFUL_RESPONSE_LINES
- Ambrose trust -= 4
- university_social += 1

sharp:
- return line from AMBROSE_SHARP_RESPONSE_LINES
- Ambrose trust -= 10
- university_social += 2

Clamp all values 0–100.

These are modest reputation shifts.
Do not make them feel like a full morality system.

NPC ENGINE — update src/engine/npcEngine.ts
═══

Do not refactor existing working logic.
Only extend it.

Update getNPCProfile() and talkToNPC() to support kilvin and ambrose
using the new NPC_PROFILES entries.

Kilvin talking rules:
- If topic is 'work', he should direct the player toward practical work, safety, or discipline
- If topic is 'sympathy', he should answer in a way that emphasises care, cost, and good sense — not theory-dump
- If topic is 'materials', he may speak practically about what is useful
- If topic is taboo, he should shut it down briefly and firmly

Ambrose talking rules:
- If topic is null, return a greeting that already carries disdain
- If topic is 'archives' or 'rank', he should respond in a way that leans on status and exclusion
- If topic is 'kvothe', he should be cutting, not melodramatic
- If topic is taboo, he should deflect contemptuously

Do not alter parseNPCCommand() unless needed to support:
  ask kilvin about work
  ask kilvin about sympathy
  ask ambrose about archives
If the existing parser already handles this, leave it alone.

═══
ACTIONS — update src/engine/actions.ts
═══

Add these commands to dispatch():

  ask kilvin for work
    -> call askKilvinForWork(state, db)
    -> return result message and newState

  work fishery / work at fishery
    -> call workFisheryShift(state, db)
    -> return result message and newState

  ignore ambrose
    -> call respondToAmbrose('ignore', state, db)
    -> return result message and newState

```
  answer ambrose carefully
    -> call respondToAmbrose('careful', state, db)
    -> return result message and newState

  answer ambrose sharply
    -> call respondToAmbrose('sharp', state, db)
    -> return result message and newState

Keep all existing commands from Prompts 1–3 intact.

After any successful Fishery shift:
  - if tuition is not yet paid, call checkTuitionDeadline() as per Prompt 2
  - return only the work result line
  - do not append a full renderLocation() automatically

After Ambrose responses:
  - return only the social line
  - do not auto-render the location

═══════════════════════════════════════════════════════
TIME / DAILY RESET — update src/engine/time.ts and economy.ts
═══════════════════════════════════════════════════════

Prompt 3 already modified advanceTime() and sleep().
Keep those changes. Add only the following:

In advanceTime():
  - When the time cycle passes from night -> morning,
    reset:
      fishery_state.approved_today = false
      fishery_state.shifts_completed_today = 0

Do not reset last_approval_day.

In economy.ts sleep():
  - After a successful sleep, also reset:
      fishery_state.approved_today = false
      fishery_state.shifts_completed_today = 0

Do not change any other economy behaviour from Prompt 2 or recovery
behaviour from Prompt 3.

═══════════════════════════════════════════════════════
TESTS — tests/socialEngine.test.ts
═══════════════════════════════════════════════════════

Import:
  askKilvinForWork,
```

  workFisheryShift,
  respondToAmbrose,
  getNpcTrust
from src/engine/socialEngine.ts.

Also import:
  initDefaultPlayerState from src/engine/state.ts
  runMigrations from src/db/schema.ts
  runSeed from src/db/seed.ts
  Database from better-sqlite3

Use an in-memory SQLite DB:
  new Database(':memory:')
Call runMigrations() and runSeed() before tests.

Tests:

  getNpcTrust:
    1. missing NPC id returns 50
    2. existing ambrose trust from initDefaultPlayerState returns 20

  askKilvinForWork:
    3. at wrong location returns denial and unchanged state
    4. at university_fishery_outer while healthy sets approved_today true
    5. at fishery with fatigue 90 returns denial and approved_today false

  workFisheryShift:
    6. without approval returns "Kilvin has not set you to work."
    7. with approval returns increased money_drabs and shifts_completed_today 1
    8. after 2 completed shifts, third attempt returns shift-complete denial
    9. successful shift advances time by 1 step

  respondToAmbrose:
    10. when Ambrose absent returns unchanged state
    11. ignore ambrose lowers ambrose trust
    12. careful response increases university_social by 1
    13. sharp response lowers ambrose trust more than careful response

Add 2 tests to existing tests/npcEngine.test.ts:

  8. talkToNPC('kilvin', 'work', stateAtFishery, db) returns non-empty string
  9. talkToNPC('ambrose', 'archives', stateAtArchivesExterior, db)
     returns non-empty string

═══
ARCHITECTURE REMINDERS
═══

These rules from Prompt 1 still apply:

- narration/* never reads from or writes to the DB
- narration/* never mutates PlayerState
- engine/* owns all state changes
- repl.ts saves state exactly once per turn
- No `any` types
- No external API calls
- No unused imports or files

socialEngine.ts is an engine-layer file.
It may read from the DB.
It must not call narrator methods directly.

═══
SUCCESS CRITERIA
═══

This prompt is complete when:

- `talk to kilvin` at the Fishery returns a voice-true line
- `ask kilvin for work` at the Fishery while healthy sets approval
- `work fishery` earns 2–3 drabs, advances time, and increases fatigue
- `ask kilvin for work` while exhausted or injured is refused
- `talk to ambrose` at the Archives exterior returns a hostile line
- `ignore ambrose` changes his trust but does not change university_social
- `answer ambrose carefully` increases university_social slightly
- `answer ambrose sharply` worsens Ambrose trust more severely
- Fishery approval resets on the next morning
- All new tests pass alongside the earlier prompts' tests
- No network calls occur at any point
- No refactoring of working Prompt 1–3 code

==========
END OF PROMPT 4
==========